

2.37. Les trois tailles côte à côte

ROND_8, ROND et ROND_32 sur le même écran : trois dessins, trois fonctions, trois tailles.

1. Ce qu'il faut comprendre

Les trois ronds ensemble, du plus petit au plus grand : ROND_8 avec `sprite()`, ROND avec `sprite16()`, ROND_32 avec `sprite32()`. Ce sont **trois dessins différents**, chacun dans son fichier.

Les lutins ne doivent pas se chevaucher : le petit prend le lutin 0 ; le moyen les lutins 1 à 4 (il en faut 4) ; le grand les lutins 5 à 20 (il en faut 16). C'est pourquoi on écrit `sprite16(1, ...)` et `sprite32(5, ...)`.

La console a une limite : 10 lutins au plus sur une même ligne de l'écran. Ici, sur les lignes où les trois ronds se trouvent, il y en a $1 + 2 + 4 = 7$. Ça passe.

Le nombre sous chaque rond dit son côté en pixels : 8, 16, 32. Chaque taille a **deux fois** le côté de la précédente, et **quatre fois** ses pixels ($8 \times 8 = 64$, $16 \times 16 = 256$, $32 \times 32 = 1\,024$).

Essaie : échange les places du petit et du grand rond.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <sprite16>` → [35.2. La fonction sprite16\(\) — un personnage de 16 × 16](#)
- `#include <sprite32>` → [35.4. La fonction sprite32\(\) — un grand personnage de 32 × 32](#)
- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1206 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	546 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	58 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	416 o	← CopierTuiles — 5 lettres sur 43, et 21 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins
"8"	1 o	← texte() — un texte gravé
"16"	2 o	← texte() — un texte gravé

```
"32"                2 0 ← texte() – un texte gravé
Lettres gravées : 12368
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
  sprite()   × 21   503 0 sur place, dans main()
  texte()    ×  3    33 0 sur place, dans main() + EcrireTexte (10 0)
  image()    ×  1     6 0 sur place, dans main() + AttendreImage (28 0)
  sprite16() ×  5     0 0 sur place, dans main()
  sprite32() ×  1     0 0 sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- 2.37 Les trois tailles côte à côte ----
// Le même rond en 8 × 8, 16 × 16 et 32 × 32.

#include <texte>           // texte() : écrire un mot
#include <sprite>          // sprite()   : un dessin de 8 × 8
#include <sprite16>        // sprite16() : un dessin de 16 × 16
#include <sprite32>        // sprite32() : un dessin de 32 × 32
#include <Tuile>           // Tuile : 8 × 8
#include <Perso>           // Perso : 16 × 16 ou 32 × 32
#include "tuile_ROND_8.cpp" // le petit rond
#include "perso_ROND.cpp"   // le rond du 2.01
#include "perso_ROND_32.cpp" // le grand rond
```

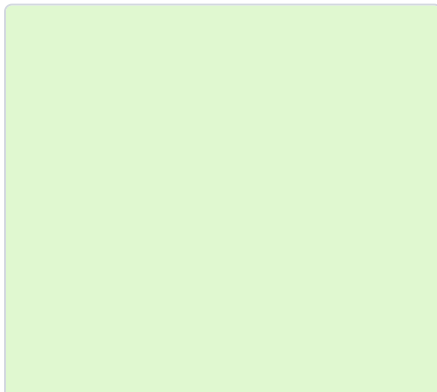
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  sprite(0, 24, 60, ROND_8);    // le lutin 0
  sprite16(1, 56, 56, ROND);    // les lutins 1 à 4
  sprite32(5, 104, 48, ROND_32); // les lutins 5 à 20
  texte(3, 11, "8");           // sous le petit
  texte(7, 11, "16");          // sous le moyen
  texte(15, 11, "32");         // sous le grand

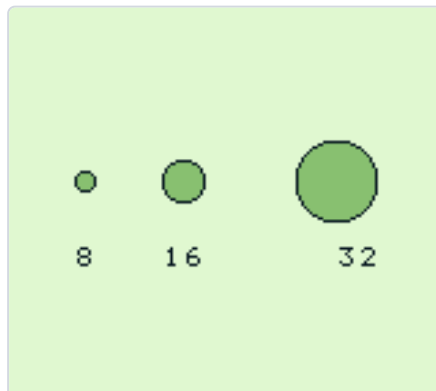
  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois ronds de plus en plus grands, et 8, 16, 32 écrits dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le petit rond en (24, 60)
- ✓ le rond du 2.01 en (56, 56)
- ✓ le grand rond en (104, 48)
- ✓ les trois tailles sont écrites

Ensuite : **2.38. Grossir un dessin : ×2** — série 2 : Les formes géométriques. · [gameboy3](#) — écrire une cartouche Game Boy en C++